Odess: Speeding up Resemblance Detection for Redundancy Elimination by Fast Content-Defined Sampling

Xiangyu Zou†, Cai Deng†, Wen Xia†‡, Philip Shilane*, Haoliang Tan†, Haijun Zhang†, and Xuan Wang†
† *Harbin Institute of Technology, Shenzhen* * *Dell Technologies*
‡ *State Key Laboratory of Computer Architecture, Institute of Computing Technology, Chinese Academy of Sciences*

2021 IEEE 37th International Conference on Data Engineering (ICDE) | 978-1-7281-9184-3/20/$31.00 ©2021 IEEE | DOI: 10.1109/ICDE51399.2021.00048

***Abstract*—Multiple data reduction techniques have been investigated to lower storage costs for a wide variety of customers. In this work, we focus on similarity-based delta compression, which calculates and stores the difference of very similar, but non-duplicate, chunks in storage systems. Delta compression is often implemented along with deduplication and has been shown to achieve a much higher compression ratio.**

Currently, the N-Transform method is the most popular and widely-used approach to generate features for data content (e.g. chunks) to detect similar candidates (and then apply delta compression). For delta compression systems, though, the throughput of N-Transform is often the bottleneck. Finesse is a high throughput variant of N-Transform, but it suffers from lower detection accuracy and compression ratio. The computation overhead of N-Transform consists of two parts: calculating the rolling hash across data and applying time-consuming transforms on each hash. In this work, we propose Odess, a fast resemblance detection approach, that uses a novel Content-Defined Sampling method to generate a much smaller proxy hash set and then applies transforms on this small hash set. This reduces the calculations in the transform step from being the bottleneck. Meanwhile, Odess also leverages the faster Gear hash to generate rolling hashes. Thus, Odess greatly reduces the computational overhead for resemblance detection while achieving high detection accuracy and high compression ratio.

Our evaluation results show that Odess is $\sim 5.4\times$ (Finesse) and $\sim 26.9\times$ (N-Transform) faster (on average) at generating features for resemblance detection. When considering an end-to-end data reduction storage system, Odess increases throughput by $\sim 1.36\times$ (Finesse) and $\sim 2.76\times$ (N-Transform) while maintaining the compression ratio of N-Transform and increasing the compression ratio $\sim 1.22\times$ over Finesse.

I. Introduction

The amount of digital data is growing explosively, a recent report from IDC [1] suggests that the total size of worldwide digital information is about 33 ZB in 2018, and that size will be about 175 ZB in 2025. As a result of this "data deluge", efficiently eliminating redundancy to reduce data storage costs became one of the most challenging and important tasks in mass storage systems, and can generally achieve a compression ratio of $10\times \sim 30\times$ [2].

To eliminate redundancy, several compression techniques have been proposed. Among the approaches, general compression [3], [4] has a slow performance or impractical memory requirements for large scale data, though it has the potential to remove fine-grained redundancies. Deduplication [5], [6], [7], [8] is a widely studied alternative that identifies and removes coarse-grained redundancies when identical chunks of data (e.g. 8KB in size) are replaced with references, though it misses redundancies smaller than the supported chunk size. Delta compression [9], [10] combines the advantages of coarse- and fine-grained redundancy reduction by detecting very similar items across a large storage system and then eliminating redundancy among them.

Delta compression has been widely applied in many applications, including databases [11], p2p systems [12], [13], backup storage systems [10], [14], remote synchronization [15], [16], memory management [17] and so on. For storage systems, the approach is to usually first detect the redundant data candidate (called resemblance detection), and then remove the redundant data relative to a candidate by using a delta encoding technique [18]. Here resemblance detection is always the key step in delta compression's workflow [14].

All of the state-of-the-art approaches for resemblance detection [11], [12], [13], [14], [15], [16], [17], [19], [20], [21] follow a similar framework of obtaining features from each to-be-detected item (a file or a chunk) and estimating the similarity of two items based on comparing their features. These approaches can be generally divided into three steps: *Characterizing, Selecting, and Indexing.* ① In the *Characterizing* step, a series of content characteristics of an item will be collected (i.e. hash values of all subchunks or all sliding windows). ② In the *Selecting* step, some of the generated characteristics will be sampled as the features of an item, where sampling techniques include "Fixed Position" [17], "Top-k" [11], [12], [13], [15], [19], [20], [21], "Top-1 on each linear transforms" [16], [22], [23], and "Top-1 on each subchunk" [14]. ③ In the *Indexing* step, similar items will be identified based on their features. To simplify this process, the features are organized for indexing, such as using the techniques of "Super-Feature" [23], "Bloom Filter" [15], "Sorting" [21], etc. Among these three steps, the first two steps focus on calculating and selecting features for items, and the last step is about finding similar candidates for delta compression according to their features.

Resemblance detection approaches usually generate several features for each item (i.e. a chunk or a file), and estimate the similarity (degree) between items according to the number of matched features. Most approaches [11], [13] generate **sequence-insensitive** features, which means features from two items may be matched out of order. For example, the ith feature of Chunk A could match the jth feature of Chunk B,

978-1-7281-9184-3/21/$31.00 ©2021 IEEE
DOI 10.1109/ICDE51399.2021.00048

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

which has similarities to the bag-of-words approach in natural language processing. Thus, for sequence-insensitive features, estimating similarity needs to calculate the intersection of the feature set for each item, which makes searching for similar candidates slow. On the other hand, N-Transform [22] is the only approach that generates **sequence-sensitive** features, meaning the ith features of items are directly compared. With the benefits of sequence-sensitive features (only considering in-position matching), N-Transform introduces the Super-Feature method, which calculates hashes over a list of features (e.g. four features) as Super-Features. When checking for similar items, Super-Features are directly searched across items, and items with any matched Super-Feature are similar candidates, which is very simple and quick. However, generating **sequence-sensitive** features involves applying several linear transforms on all the characteristics generated from the *Characterizing* step, which is computationally more expensive in the *Selecting* step [14], [24].

Finesse [14] aims at improving N-Transform for the *Selecting* step by an approximation method, which exploits the fine-grained locality inside similar items. Compared with N-Transform, Finesse achieves much higher speed ($5\times$ [14]) by avoiding the calculation of linear transforms in the *Selecting* step, but it generates sequence-insensitive features. Finesse applies Super-Feature (which usually requires sequence-sensitive features) in the *Indexing* step for faster searches for similar candidates, although Finesse's features are sequence-insensitive. Thus, it leads to errors in detecting similar candidates (a lower detection efficiency) and also a lower compression ratio.

Thus, to combine the advantages of the above two approaches, we try to design an approach which generates sequence-sensitive features at a high speed. Specifically, we focus on reducing the calculations of N-Transform, and several observed properties guide our research. The calculation of N-Transform mainly includes two parts (*Characterizing* and *Selecting*), and we optimize them separately. For the *Characterizing* step, the traditional Rabin rolling hash requires many calculations on each window update, and we replace it with a faster rolling hash, called Gear [25]. For the *Selecting* step, its calculation is related to the size of the item, and we introduce a novel **Content-Defined Sampling** method before the *Selecting* step. This generates a smaller proxy (i.e., a sampled set) by consistently sampling the content-defined hash values (i.e., Gear hashes) and then applying linear transforms on the proxy, thus the calculation for the *Selecting* step is dramatically reduced. At the same time, we verified that, compared with N-Transform, our Content-Defined Sampling method with a reasonable sampling rate (e.g., 1/128) has only slightly impact on the detecting efficiency, in theory and evaluation.

Finally, we propose a fast resemblance detection approach, named Odess, which generates sequence-sensitive features by using a novel Content-Defined Sampling method on the fast Gear rolling hash. Generally, Odess achieves three benefits:

- A much faster speed of generating sequence-sensitive features compared with the state of the art.
- Follows Broder's Theorem [22], which ensures high detection accuracy and producing sequence-sensitive features.
- It is more scalable than previous approaches by using Content-Defined Sampling: e.g., can efficiently support many more features for better detection efficiency.

Compared with Finesse, Odess generates features about $5.5\times$ faster and is more accurate with resemblance detection. Compared with N-Transform, Odess is about $26.9\times$ faster at feature generation while maintaining a comparable similarity detection efficiency. In addition, we also evaluate approaches in a real-world application: a data reduction system with deduplication and delta compression, in which the feature generating speed impacts the throughput of the whole system as well as the compression ratio. Compared with Finesse, the system with Odess achieves about $1.36\times$ faster throughput and $1.22\times$ higher compression ratio; Compared with N-Transform, a system with Odess has about $2.76\times$ faster throughput and nearly the same compression ratio.

II. Background and Related Works

A. Background

Redundancy elimination is an important technique to control storage costs, and there are generally three approaches: general compression, data deduplication, delta compression.

General compression is designed for reducing redundancy data at the byte/string level by using traditional algorithms, e.g., dictionary coding [3], [4] and Huffman coding [26]. Because of fine-grained redundancy searching, general compression usually achieves a good compression ratio. But it also has several weaknesses: For large-scale storage systems, the compression/decompression speed of general compression is not high enough for processing such amounts of data; Also, because general compression has a window within which it can find repeated patterns (the window size is kilobytes to megabytes typically), general compression cannot identify data redundancies that are distant from each other.

Data deduplication [5] is able to identify duplicate regions across an entire storage system, though at a coarse granularity. Data deduplication works by splitting data into chunks (or fixed-sized blocks) and replacing duplicate chunks with references. An index of content hashes and other mechanisms are used to identify duplicates. Compared with general compression, deduplication has a processing (i.e., compressing) speed up to the GB/s level and is practical for large storage systems. But, because deduplication cannot identify redundancies smaller than the chunk size, it cannot remove redundancies between similar, non-identical chunks.

Delta compression [9], [16] bridges the gap between data deduplication and general compression and is usually implemented along with both to maximize the data compression ratio. Items (i.e. deduplication unit, a file or a chunk) are represented by features, and the similarity of two items is estimated based on comparing their features. When similar items are found, delta encoding [18] will be applied to find the byte-wise diff, which is typically much smaller for highly similar items.

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

TABLE I
APPROACHES FOR RESEMBLANCE DETECTION

Approaches	Characterizing	Selecting	Indexing	Type of feature
Coarse-grained Top-k [11], [13], [19]	coarse-grained	Top-k	Bucketing	sequence insensitive
Fine-grained Top-k [12], [20]	fine-grained	Top-k	Bucketing	sequence insensitive
Coarse-grained BF [15]	coarse-grained	Top-k	Bloom Filter	sequence insensitive
Coarse-grained Sorting [21]	coarse-grained	Top-k	Sorting	sequence insensitive
N-Transform [16], [23]	fine-grained	Top-1 on each linear transform	Super Feature	sequence sensitive
Fixed-position [17]	fine-grained	Fixed Position	Super Feature	sequence insensitive
Finesse [14]	fine-grained	Top-1 on each subchunk	Super Feature	sequence insensitive

This approach can eliminate fine-grained redundancy across an entire storage system. It has been applied in many works from different domains, e.g. storage systems [14], [16], [23], databases [11], [19], p2p systems [12], [13], memory [17], etc.

B. Existing Resemblance Detection Approaches

As delta compression is able to maximize redundancy elimination among similar chunks in storage systems (compared with general compression and deduplication), we focus on resemblance detection for delta compression in this work.

Many approaches of resemblance detection have been proposed that differ in important ways, which is summarized in Table I. All of these approaches generate features from each item and then compare features to estimate similarity. **Detection accuracy** reflects the approximation degree between the actual similarity and the estimated similarity. Among these approaches, they share three common steps:

1) **Characterizing**: characteristics of the item (e.g., hash values of all subchunks or all sliding windows) are generated from an item.
2) **Selecting**: an extracting method is applied to reduce the scale of the characteristics, thus features (a.k.a. signature or other terms in some papers) are extracted from characteristics generated from the 1st step.
3) **Indexing**: item's features are added to a similarity index.

Coarse-grained Top-k is used in many works [11], [13], [19]. Two works [13], [19] apply this approach to detect similar files, and another one [11] applies this approach to detect similar records in databases. The *Characterizing* step for this approach is separating a to-be-detected item into several blocks with Content-Defined Chunks (CDC), and calculating hash values for each block. All hash values consist of the characteristics of an item, and similar items will share some of the same characteristics, which means they are likely to share some of the same blocks. Because this *Characterizing* identifies commonality at the block level, this is a coarse-grained approach. The *Selecting* step for this approach selects up to the maximum k characteristics as features, which we called Top-k. Similar items will share some or all of the features, and items that have more features in common tend to have a higher similarity degree. This kind of feature is **sequence-insensitive**, because the order of features is ignored, and matched features may or may not have positional correspondence as shown in Figure 1. Therefore, the intersection and union of features are required to calculate the Jaccard Similarity metric:

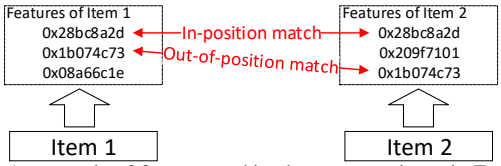


Fig. 1. An example of feature matching between two items in Top-k. Note that these features are sequence-insensitive, which means in-position and out-of-position matches are both possible and complicate resemblance detection.

$$Similarity(item1, item2) = \frac{|\{Features\ of\ item1\} \cap \{Features\ of\ item2\}|}{|\{Features\ of\ item1\} \cup \{Features\ of\ item2\}|}. \quad (1)$$

Thus, features' matching between items is complex, which makes indexing mechanisms are also complex. The usual *Indexing* step for this approach is Bucketing, in which each feature maps to a bucket. $Bucket(f)$ records all the items which includes the feature f. When looking up a similar candidate for a given item k, we first generate the features of k (such as f_1, f_2, f_3), next we check the corresponding buckets ($Bucket(f_1)$, $Bucket(f_2)$, $Bucket(f_3)$), and finally we select the item that has the most matching features from these buckets. Thus, finding candidates needs to traverse all the corresponding buckets, which is time-consuming. Meanwhile, this indexing is memory-intensive since all of the features need to be saved in buckets.

Fine-grained Top-k has been applied in previous work [12], [20]. The first step in this approach is running a rolling hash method on the to-be-detected item, sliding the window of rolling hash byte-by-byte, and recording the hash value of each sliding window. Because this kind of *Characterizing* step generates characteristics at the byte/string level, we call it fine-grained style *Characterizing*. The second and the third steps of this approach are also Top-k and Bucketing, and the drawbacks are the same as the Coarse-grained Top-k approach.

Coarse-grained BF [15] has been applied to detect similar chunks at a large granularity (e.g.16MB). The first step is chunk-based (Coarse-grained: at the chunk level) and the second step is Top-k, which is the same as the first approach. The novel part of this technique is introducing a Bloom filter [15] in the *Indexing* step. After features are generated with chunk-based *Characterizing* and Top-k style *Selecting*, they will be inserted into a Bloom filter. Therefore, each item is represented by a Bloom filter, and the similarity of two items can be estimated with the hamming distance between their Bloom filters. Compared with 'Bucketing', this indexing solution reduces the memory cost, but it also has serious shortcomings. First, because of the hash collisions, we can not acquire the exact similarity from Bloom filters. Second, there is no straightforward way to index Bloom filters (unlike features), so finding a similar item may involve comparing against all the previous Bloom filters.

Coarse-grained Sorting [21] focuses on offline similar file detection. The 1st and the 2nd steps also use chunk-based *Characterizing* and Top-k *Selecting*. Because it is designed for offline situations, sorting is used to estimate the similarity

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

relationship between files. Features of all files in the system are recorded in a feature-file and an external sort is applied to all the features. After sorting, identical features will be consecutive and the corresponding files are likely similar. This kind of *Indexing* is also time-consuming and thus is applied offline in resemblance detection.

N-Transform was first proposed by Broder [22], and it has been applied in several works [16], [23], which focuses on detecting similar chunks (about 8KB in general). It is based on Broder's Theorem that requires several predefined linear transforms. In the *Characterizing* step, this approach follows the fine-grained style. In the *Selecting* step, N-Transform, which is very different from the above approaches, calculates linear transforms on hash values of each sliding window and records the minimum values of each linear transform's result as features. Features generated by this kind of *Selecting* are **sequence-sensitive**. According to Broder's Theorem, the probability of the ith feature from two items being identical is equal to the Jaccard Similarity [27] of two items. Thus, we only need to consider positional matches of features in N-Transform. Therefore, this kind of feature is friendly to indexing. Finally, if the number of the features for an item is n and the number of matched features is k, the Jaccard Similarity can be simply estimated by

$$Similarity(Item1, Item2) = \frac{k}{n}. \tag{2}$$

The *Indexing* step of N-Transform is Super-Feature, first proposed by Kulkarni et al. [23], in which features are grouped into several **Super-Features**. For example, if we have features f_1, f_2, f_3, f_4, they can be grouped into two Super-Features as $SF_1 = hash(f_1, f_2)$ and $SF_2 = hash(f_3, f_4)$. The numbers of features and Super-Features are configurable, and two items sharing at least one Super-Feature will be regarded as similar items. Compared with other indexing approaches, Super-Feature leads to much faster similarity search and lower memory costs. N-Transform has downsides though, as the linear transforms make it slow in the *Selecting* step.

Fixed-Position [17] is designed for redundancy detection and elimination in memory. When virtual machines run on a physical machine, memory pages of different virtual machines may be identical or very similar. This approach picks data blocks in predefined positions within pages and calculates hash values for each data block. Because the positions are random, the characterization of this approach could be regarded as fine-grained, and we call the *Selecting* step of this approach as fixed position, but it suffers from not only the known 'boundary-shift' problem [28], which means the position of identical contents in two items may be shifted (due to content modifying, inserting, etc.), but also poor detection efficiency and thus low compression ratio. Note that the *Indexing* used in this approach is also "Super-Feature".

Finesse [14] is designed to significantly improve the *Selecting* performance of N-Transform. The first step of Finesse is also fine-grained style *Characterizing*. The second step of Finesse separates the item into several subchunks and records the minimum hash value in each subchunk as features, which generates sequence-insensitive features. The third step of Finesse also follows the idea of the Super-Feature method, and to alleviate the disadvantages of sequence-insensitive features, a sorting step is applied before grouping Super-Features. However, the accuracy of Finesse is still not as high as N-Transform, and we classify Finesse as sequence-insensitive, despite the sorting step, since sorting features does not overcome insertion and deletion changes.

In summary, among approaches we reviewed, N-Transform is the only option with sequence-sensitive features (simple and fast to match) and high detection accuracy, which makes it the most attractive option to pursue resemblance detection [16], [23], [24], [29]. But generating sequence-sensitive features is time-consuming in N-Transform. In the following subsections, N-Transform and its improved approach Finesse will be presented in greater details to motivate our work.

C. N-Transform Details

As mentioned above, N-Transform is the most popular method for detecting very similar chunks for redundancy elimination [16], [23], [29] due to generating sequence-sensitive features.

N-Transform, as Figure 2 shows, relies on Broder's Theorem [22], [30], which states that the probability of the two sets A and B having the same minimum hash element is the same as their Jaccard similarity coefficient [27], as detailed below:

Broder's Theorem. *Consider two sets A and B, with H(A) and H(B) being the corresponding sets of the hashes of the elements of A and B respectively, where H is chosen uniformly and randomly from a min-wise independent family of permutations. H maps an element in the set to an integer. Let min(S) denote the minimum element of the set of integers S. Then:* $Pr[min(H(A)) = min(H(B))] = \frac{|A \cap B|}{|A \cup B|}$

Based on this theorem, Broder proposed a resemblance detection approach that extracts a fixed number of features from a chunk. As introduced in Section II-B, N-Transform also includes three steps: *Characterizing*, *Selecting* and *Indexing*.

In the *Characterizing* step, rolling hash (often Rabin [31]) is applied to generate a series of fingerprints, and these fingerprints make up a *characteristic set* (i.e. sets A or B in Broder's Theorem).

Linear transforms (denoted as $LT()$) are used as the maps (i.e. $H()$) in Broder' Theorem and to generate features in N-Transform. For example, Feature i of a chunk (length = L), is uniquely generated with a corresponding linear transform, which is defined as

$$LT(x) = m_i \times x + a_i, \tag{3}$$

where m_i and a_i are predefined random values. The Feature i with a sliding window size of 32 bytes is acquired as $Feature\ i = Min_{j=1}^{L}(m_i * Rabin_j + a_j)\ mod\ 2^{32}$.

N-Transform generates features for each chunk, and their i^{th} features are all produced by the same linear transform. Thus, the linear transform could be regarded as $H()$ in

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

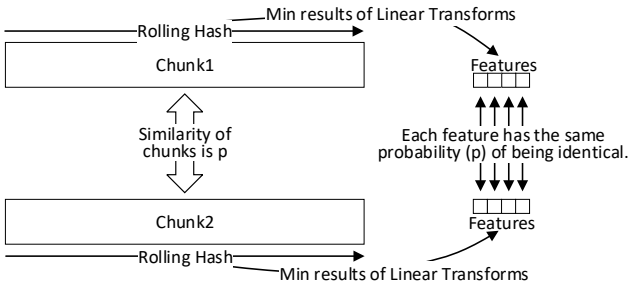


Fig. 2. General description of N-Transform's principle.

Broder's Theorem, and the similarity of two chunks is equal to the probability of the minimum values of transformed *characteristic sets* (i.e., $LT(A)$ and $LT(B)$) being the same.

Just like we can estimate the probability of the result of a coin flip by by tossing it multiple times, here, we use multiple linear transforms to generate multiple features and observe the number of matches to estimate the similarity between chunks. Each feature can be regarded as a series of independent, repeated trials. Thus, we can summarize that **the principle of N-Transform is running independent repeated trials on the whole chunk with different linear transforms**. Since Border's Theorem only defines matching with the same linear transform, N-Transform generates **sequence-sensitive** features.

Therefore, the *Selecting* step of N-Transform runs, as shown in Figure 3. The 'Rolling Hash' is applied to the 'Data Chunk' and generates a series of 'FP values'. *Characteristics set* consists of 'FP values', and several predefined linear transforms are applied to it, which generates several series of 'Transformed FP'. After that, N-Transform records the minimum value of several series of 'Transformed FP' as features. When estimating the similarity of two chunks, we refer to features from two chunks generated by the same linear transform as **"a pair of features"**. Then, if N-Transform produces n features (from n linear transforms) for each chunk, and there are k pairs of features from two chunks that are identical, we can get the **Maximum Likelihood Estimation** of the actual similarity between two chunks as $\frac{k}{n}$.

Algorithm 1: Extracting features in N-transform.

Input: chunk content, Str; length of the chunk, L; random value pair for linear transformation, m_i and a_i
Output: N features, Feature[N]
$Feature[0, ..., N-1] \leftarrow 0$;
for $m = 0;\ m < L;\ m++$; **do**
  $FP \leftarrow RabinFunction(Str, m)$;
  for $i = 0;\ i < N;\ i++$; **do**
    $Transform[i] \leftarrow (m_i \times FP + a_i) mod 2^{32}$;
    if $Feature[i] \geq Transform[i]$ **then**
      $Feature[i] \leftarrow Transform[i]$;
return Feature[N];

Algorithm 1 provides a detailed pseudo-code implementation of extracting features by N-transform. Note that features in N-Transform are sequence-sensitive, because features can only be compared to features generated by the corresponding linear transformation function.

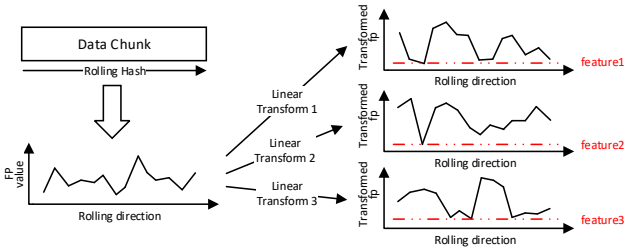


Fig. 3. An example of extracting three features in N-Transform.

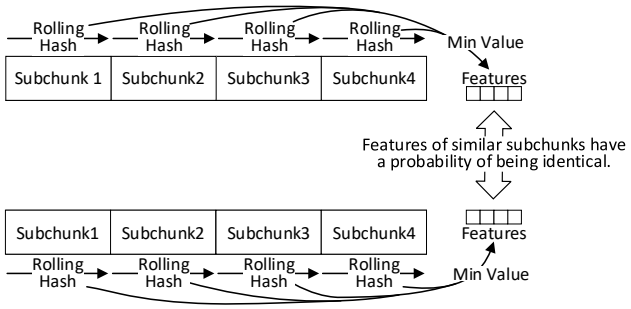


Fig. 4. General description of Finesse's principle.

In the *Indexing* step, Features are grouped into a small number of Super-Features as explained in Section II-B, and Super-Features may be indexed by a hashtable or other structures. Finally, if two chunks share a same Super-Feature, they are likely to be highly similar.

D. Finesse Details

Zhang et al. [14] proposed a method named Finesse, which improved the feature-generation throughput of N-Transform. The main idea of Finesse is consistent with the backup stream locality, which is observed by many studies [32], [33], [34], [35], [36], [37] and refers to the fact that the corresponding subchunks of chunks and their features appear in the same order among the similar chunks with a very high probability.

Finesse also can be partly explained by Broder's Theorem, as shown in Figure 4 and Figure 5. As we discussed in Section II-C, N-Transform applies a series of independent repeated trials on chunks to estimate the similarity. Finesse, which is based on the assumption that the differences between similar chunks are evenly distributed, runs another kind of independent repeated trials. It first splits a chunk into several equal-sized subchunks. According to the assumption, Finesse considers that the similarity between two chunks' $i-th$ subchunks should be equal to the similarity of two whole chunks. After that, Finesse introduces a trivial linear transform ($LT(x) = 1 \times x + 0$, i.e. $m_i = 1$ & $a_i = 0$ in Equation 3) and applies Border's Theorem on each pair of subchunks. A rolling hash generates a *characteristics set* for each subchunk, and the trivial linear transform is applied to the *characteristics set*. Then, the minimum values of each transformed *characteristics set* are picked as features. Because the assumption suggests that all pairs of corresponding subchunks from two chunks share the same similarity, all pairs of features from

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

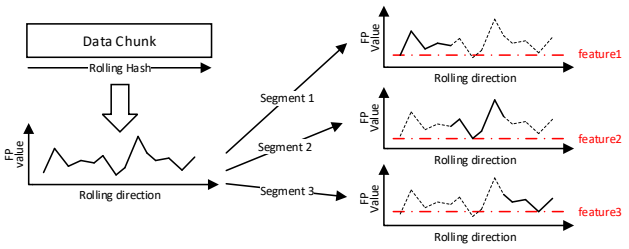


Fig. 5. An example of extracting three features in Finesse.

corresponding subchunks also have the same probability to be the same. Thus, the similarity of the two chunks also could be estimated as N-Transform does. Because there are no additional calculations for linear transforms in Finesse, it achieves much higher performance than traditional N-transform.

Moreover, we can also summarize that **the principle of Finesse is running independent repeated trials on different subchunks with a trivial linear transform**.

However, because Finesse uses fixed-sized subchunks, it is obvious that Finesse will encounter the known 'boundary-shift' problem [28] when extracting features, and the features of Finesse should be regarded as sequence-insensitive. Therefore, Finesse is different from N-Transform, and features of two chunks are not in one-to-one correspondence. For example, if feature #3 of chunk A is equal to feature #4 of chunk B, it means the corresponding Subchunk #3 is similar to Subchunk #4. The more features that are shared by chunks A and B, the higher the similarity of chunks A and B is estimated to be, and we can measure the similarity of two chunks by the number of shared features (see Equation 1).

Finesse also uses Super-Features to simplify the process of matching features. Because its features are sequence-insensitive, Finesse uses a different Super-Feature technique. For example, there are six features f_1, f_2, f_3, f_4, f_5, f_6. To form three Super-Features, Finesse begins by dividing features into two lists f_1, f_2, f_3 and f_4, f_5, f_6. Next, it sorts each list of features numerically: assuming the sorted results are f_2, f_1, f_3 and f_6, f_4, f_5. Finally, it groups Super-Features as $SF_1 = hash(f_2, f_6), SF_2 = hash(f_1, f_4), SF_3 = hash(f_3, f_5)$. The sorting step alleviates the negatives from sequence-insensitive features, and makes applying Super-Feature is feasible. However, Finesse still suffers from the 'boundary-shift' issue.

In summary, Finesse accelerates the *Selecting* step since no linear transforms are required, but reduces the detection accuracy due to generating sequence-insensitive features.

III. Motivation and Design

From our review of existing approaches, it can be seen that N-Transform has a better detection accuracy but suffers from slow feature generating speed; Finesse has a higher feature generating speed but suffers from a worse detection accuracy. Thus, our focus is on a new approach that achieves both a high feature generating speed and a better detection accuracy.

As discussed in Section II-C, in order to speed up N-Transform, we need to reconsider the calculation overhead of all three steps of resemblance detection in N-Transform:

Algorithm 2: Extracting features in Finesse.

Input: chunk content, Str; length of the chunk, L;
Output: N features, Feature[N]
$subChkSize \leftarrow L/N$;
$Feature[0, ..., N-1] \leftarrow 0$;
for $m = 0;\ m < L;\ m++$; **do**
  for $i = 0;\ i < subChkSize;\ i++$; **do**
    $FP \leftarrow RabinFunction(Str, m * subChkSize + i)$;
    if $Feature[n] \geq FP$ **then**
      $Feature[n] \leftarrow FP$;
return Feature[N];

Characterizing and *Selecting* are very time-consuming for generating fine-grained sequence-sensitive features and there are opportunities to speed up these two steps, while super-feature based *Indexing* is very efficient for looking up similar candidates and there is no need to change it. Therefore, our goal is to reduce calculations in the first two steps.

A. Reducing Calculations for the First Two Steps

For the *Characterizing* step, previous work has discussed the calculation cost of Rabin rolling hash [25], [38]: Rabin rolling hash [31], [28] requires about 5 calculating operations and 2 memory accesses each time the window is slid by one byte. A much faster approach has been proposed: Gear rolling hash [25], which requires nearly half of the operations per byte of data. Thus, replacing Rabin with Gear could decrease the calculation cost in the *Characterizing* step.

The *Selecting* step, as discussed in Section II-C, usually extracts the characteristics generated in the previous phase. In N-Transform, a special extracting method based on Broder's Theorem [22] is used to produces sequence-sensitive features, but it requires calculating many linear transforms on each characteristic, which is a large calculation workload.

The overhead of the linear transforms' calculation is related to the size of to-be-detected items. For example, if 12 linear transforms are required for generating 12 features, and the size of to-be-detected items is N, the total calculation in *Selecting* is $36N$ (multiply, add, and compare for each characteristic). The calculation of linear transforms is difficult to reduce, thus, we consider whether we could reduce the size of "to-be-detected items".

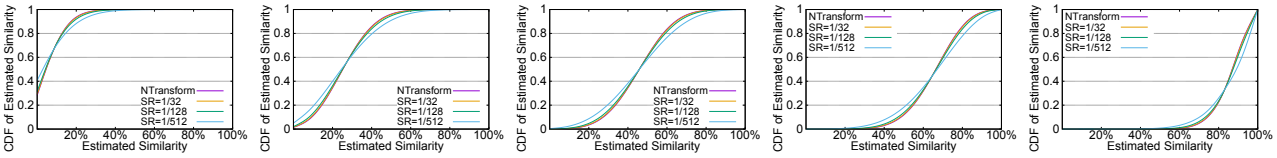
Broder's Theorem suggests that the probability that each pair of features is the same depends on the similarity of two items, and is not dependent on the span of data covered by the features. Thus, if we build **proxies**, which have a smaller size, to replace to-be-detected items in the *Selecting* step, we can apply linear transforms on proxies and the total calculation in *Selecting* is reduced. We next address the problem of generating proxies for each to-be-detected item.

B. Generating Proxies by Content-Defined Sampling

The simplified proxies must satisfy following preconditions:

- **Certainty.** Identical items must generate identical proxies. For example, random sampling can not promise this.
- **Retention of similarity.** Similar to-be-detected items must generate similar proxies, and the similarity degree of

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.



(a) Actual Similarity=10% (b) Actual Similarity=30% (c) Actual Similarity=50% (d) Actual Similarity=70% (e) Actual Similarity=90%

Fig. 6. Cumulative distribution function of estimated similarity for N-transform and our Content-Defined approach when varying p (Actual Similarity) and sampling rates of 1/32, 1/128 and 1/512. We set the expected average chunk (item) size as 8KB and the required number of features as 12. The results suggest that a larger sampling rate (i.e. 1/32) is close to N-Transform while a smaller sampling rate will increase estimation errors, and 1/128 is reasonably close.

two proxies should approximate the similarity degree of their corresponding items (similar to Broder's Theorem).

- **Speed.** The generation of proxies must be fast.

Sampling is a direct roadmap for generating proxies, but according to these preconditions, many sampling methods are inapplicable, including random sampling, systematic sampling, etc. Even if we use a fixed random seed, the generated random sampling point is also fixed, and it will be similar to 'Fixed-Position', which is introduced in Section I and can not handle the 'boundary-shift' issue. Inspired by the principle of Content-Defined Chunking (CDC) [31], [38], we find that a 'Content-Defined Sampling' is feasible to satisfy the Certainty precondition.

More specifically, in CDC, we calculate fingerprints on each sliding windows, and check whether the fingerprint fp satisfies a predefined condition (e.g. 'fp & mask == 0', $mask$ is an user-specified number [38]) to set a chunking cut point. Similar to this, we could generate proxies with Content-Defined Sampling: we check each characteristic (i.e. fp) and decide whether it is in a proxy by also checking whether fp satisfies the predefined condition. Note that this condition is related to the sampling rate, which is decided by the value of $mask$. For example, if we use a condition that 'fp & 0x1c == 0', the sampling rate is about 1/8 in our Content-Defined Sampling, which is decided by the number of '1' bits in '0x1c' (i.e. three bits set to '1' causes a sampling rate of $\frac{1}{2^3}$).

C. Retention of Similarity in Content-Defined Sampling

Content-Defined Sampling satisfies Certainty and Speed as discussed in the last subsection, and we discuss whether it ensures Retention of similarity in this subsection. Specifically, we have introduced a Content-Defined Sampling method in the *Selecting* step (before applying transforms), and the estimated similarity of our approach will vary around the estimated similarity of N-Transform. Thus, we analyze estimated similarity of N-Transform and our approach to demonstrate whether our approach maintains similarity (after sampling).

As described in Section II-C, N-Transform estimates the similarity of two chunks by comparing n feature from two chunks. For example, the actual similarity of two chunks is p, the number of identical features from the two chunks is k, and the estimated similarity is s. According to Broder's Theorem, each feature has an equal probability (i.e. p) to be identical. Thus, checking whether features are matched can be treated as a sequence of n independent experiments, and the count of matched features should follow the Binomial distribution. Thus, according to the probability mass function of the Binomial distribution, we learn that when $p = p_0$, the probability of k_0 features to be matched will be

$$Pr(k = k_0 \mid p = p_0) = C_n^{k_0} p_0^{k_0} (1 - p_0)^{n-k_0}. \tag{4}$$

Here $C_n^{k_0}$ means choosing k_0 elements from n elements. Because $s = k/n$, therefore

$$Pr(s = \frac{k_0}{n} \mid p = p_0) = C_n^{k_0} p_0^{k_0} (1 - p_0)^{n-k_0}. \tag{5}$$

Thus, we get the relationship (Equation 5) between estimated similarity (i.e. s) and other specified parameters (i.e. p and n) for N-Transform.

On the other hand, we also consider the relationship between estimated similarity (i.e. s') and other specified parameters for a proxy-based approach. We denote the sampling rate as r, and the chunk size as c. Thus, we can get that the size of proxies will be $r \times c$. We also denote the number of identical elements in two proxies for two items as m, and the similarity of proxies as p'. Firstly, we consider the distribution of m: As we know, for two items that share characteristics, each element in the proxies has a probability (i.e. similarity p) to be sampled from the shared part between two items. Thus, the number m also follows the Binomial distribution, and we learn that when $p = p_0$ and $r = r_0$, the probability of $m = m_0$ is

$$\begin{aligned} &Pr(m = m_0 \mid p = p_0,\ r = r_0) \\ &= Pr(p' = \frac{m_0}{r_0 \cdot c} \mid p = p_0,\ r = r_0) \\ &= C_{r_0 \cdot c}^{m_0} {p_0}^{m_0} (1 - p_0)^{r_0 \cdot c - m_0}. \end{aligned} \tag{6}$$

Next, assuming the number of matched features is k', we further consider the relationship between estimated similarity ($s' = k'/n$) and other specified parameters for our approach:

$$\begin{aligned} &Pr(s' = \frac{k'_0}{n} \mid p = p_0,\ r = r_0) \\ &= \Sigma Pr(s' = \frac{k'_0}{n} \mid p' = p'_0) Pr(p' = p'_0 \mid p = p_0,\ r = r_0) \\ &= \Sigma Pr(s' = \frac{k'_0}{n} \mid p' = \frac{m_0}{r_0 c}) Pr(p' = \frac{m_0}{r_0 c} \mid p = p_0,\ r = r_0) \end{aligned} \tag{7}$$

Equation 4 suggests the result of $Pr(s' = \frac{k'_0}{n} \mid p' = \frac{m_0}{r_0 c})$, and Equation 6 gives the result of $Pr(p' = \frac{m_0}{r_0 c} \mid p = p_0, r = r_0)$. Thus, we get

$$\begin{aligned} &Pr(s' = \frac{k'_0}{n} \mid p = p_0,\ r = r_0) \\ &= \Sigma_{m_0=0}^{r_0 c} C_n^{k'_0} (\frac{m_0}{r_0 c})^{k'_0} (1 - \frac{m_0}{r_0 c})^{n-k'_0} C_{r_0 c}^{m_0} {p_0}^{m_0} (1 - p_0)^{r_0 c - m_0}. \end{aligned} \tag{8}$$

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

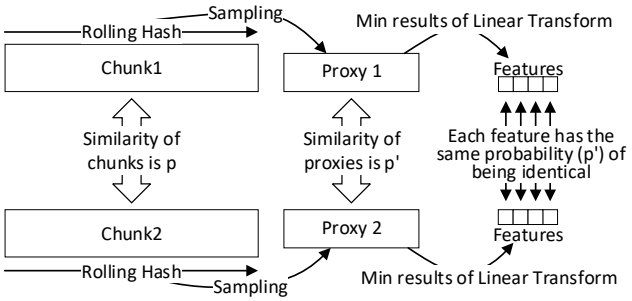


Fig. 7. General description of Odess's principle.

Finally, according to Equations 5 and 8, we study the relationship between the actual similarity and estimated similarity of approaches. Figure 6 is generated by these two equations with some specific parameters, which shows the CDF (Cumulative Distribution Function) of the estimated similarity of different approaches on different actual similarity values between two items. From the results shown in Figure 6, we first see that estimated similarity's expectation of our approach is the same as N-Transform, which leads to an inference that Inference 1: compared with N-Transform, our Content-Defined Sampling approach will generate the same similarity on average. Meanwhile, results show that a bigger sampling rate (i.e. 1/32) will lead to a result which is closer to N-Transform, and a smaller sampling rate will increase the estimation errors, which reflects another inference that Inference 2 : the sampling rate determines the robustness of our approach.

Thus, we learn that the Content-Defined Sampling method maintains similarity and all three preconditions are satisfied.

D. Implementation Details

Previous subsections suggest that Content-Defined Sampling is proven to be feasible for generating proxies, which can reduce the calculation overhead in the *Selecting* step. Based on that, we propose a new resemblance detection approach using Content-Defined Sampling, named Odess, with a much higher speed for generating sequence-sensitive features while maintaining a high detection accuracy similar to N-Transform. Therefore, **the principle of Odess is running independent repeated trials on the proxy with different linear transforms**.

The general principle of Odess is shown in Figure 7, also following the typical three steps:

1) **Characterizing:** Odess follows the fine-grained style and applies the Gear rolling hash method to quickly collect a set of characteristics (i.e. all the rolling hash values) for each item.
2) **Selecting:** Odess generates a proxy with Content-Defined Sampling for each item. After that, several predefined linear transforms are applied on the proxy to reduce calculations, and the minimum values of each linear transform will be extracted as features.
3) **Indexing:** The Super-Feature technique is used, so features are grouped into fewer Super-Features.

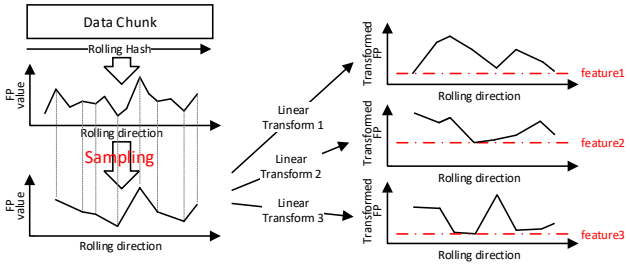


Fig. 8. An example of extracting three features in Odess.

Algorithm 3: Extracting features in Odess.

Input: chunk content, Str; length of the chunk, L; randomly value pair for linear transformation, m_i and a_i; a mask to sample, $mask$; a predefined table for computing Gear rolling hash, $Gear$
Output: N features, Feature[N]

$Feature[0, ..., N-1] \leftarrow 0$;
for $m = 0;\ m < L;\ m++$; **do**
  $FP \leftarrow (FP \ll 1) + Gear[m]$; //GearFunction
  if $FP\&mask = 0$ **then**
    for $i = 0$ *to* $N-1$ **do**
      $Transform[i] \leftarrow (m_i * FP + a_i) mod 2^{32}$;
      if $Feature[i] \geq Transform[i]$ **then**
        $Feature[i] \leftarrow Transform[i]$;
return Feature[N];

Compared with N-Transform, the calculation of Odess is reduced substantially. In *Characterizing*, the calculation of Gear is nearly half of Rabin [25]. In *Selecting*, the calculation is reduced to the sampling rate in generating Proxy, which is usually configured as 1/128, so *Selecting* of Odess has approximately 1/128 of the calculations of N-Transform. Content-Defined Sampling in generating proxy introduces more calculations (i.e. Content-Defined Sampling), but it is offset by the advantage from using the Gear rolling hash, as shown in Algorithm 3. Figure 8 also provides an example of extracting three features in Odess, which suggests our Content-Defined Sampling can well reduce the calculation of linear transforms in generating sequence-sensitive features.

IV. Evaluation

In this section, we first verify whether our approach reaches the goal of design: a much faster resemblance detection than N-Transform with a better detection accuracy and a higher compression ratio than Finesse. Then, we apply our approach in a prototype data reduction system to further demonstrate the impacts that these approaches can bring to a system.

A. Experimental Setup

We perform our evaluation on a server with two Intel Xeon Gold 6113 processors @ 2.1G HZ, 128GB memory, and four Intel S4600 1TB SSDs. The server runs Ubuntu 16.04, and the prototype data reduction system has a five-stage pipelines, including reading, chunking, hashing, deduplicating, and writing stages.

In experiments, we built a benchmark to generate data chunk pairs with varying similarity distributions to study the detection accuracy of Odess (detailed in Section IV-B). Then

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

TABLE II
WORKLOAD CHARACTERISTICS OF THE SIX TESTED DATASETS IN OUR EVALUATION. DEDUP RATIO IS REPORTED WITH A CHUNK SIZE OF 8KB.

Name	Size	Dedup Ratio	Workload descriptions
TAR	167 GB	2.08	492 tarred files from several open source projects such as GCC, Emacs, GDB, OpenJDK, Node.js, etc.
LNX	116 GB	2.00	260 versions of Linux kernel source code. Each version is packaged as a tar file
RDB	431 GB	12.58	66 backups of the redis key-value store database
SYN	452 GB	13.77	55 synthetic backups by simulating file create/delete/modify operations
VMA	138 GB	1.70	91 virtual machine images of different OS release versions, including Fedora, CentOS, Debian, etc.
VMB	314 GB	8.14	21 backups of an Ubuntu 12.04 VM image

we also build a prototype data reduction system (deduplication + delta compression) to demonstrate the impact of Odess at a system level (detailed in Section IV-C).

For the prototype experiments, six datasets are used as shown in Table II. These datasets represent various typical backup workloads for redundancy elimination, including website snapshots, source code projects, virtual machine images, database images, and a synthetic dataset. These datasets have been used in several data reduction studies [14], [39], [38].

B. Detection Accuracy on Benchmark Datasets

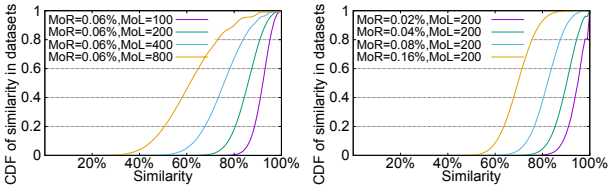
In this subsection, we evaluate three resemblance detection approaches on a benchmark, focusing on the detection accuracy as measured by two metrics, **Average Estimating Error** (lower is better) and **Standard Deviation of Estimating Error** (lower is better). **Estimating Error** is the absolute value of the difference between the actual similarity (recorded by benchmark) and the estimated similarity (calculated after detecting by the tested approaches with Equation 2). Average Estimating Error shows the general detection accuracy of approaches and Standard Deviation of Estimating Error shows the stability of approaches.

In order to better evaluate detection accuracy as similarity varies in a controlled manner, we create a benchmark instead of using existing datasets. We generate random chunks and apply a series of modifications on them to derive similar chunks in our benchmark, in which the actual similarities of these pairs of chunks are recorded. Modifications randomly occur on each byte with a probability that we call the **"modification rate"** (denoted by **MoR**). Each modification will perform an insertion, a deletion or a replace with an equal probability, and we call the length of insertion, deletion or replace as **"modification length"** (denoted by **MoL**).

Two series of benchmark datasets are generated by varying the **MoL** and **MoR** as illustrated in Figure 9(a) and Figure 9(b). Modifying the length has a larger impact on similarity distributions, causing them to be more spread than changing the modification rate.

1) *Comparison among Three Resemblance Detection Approaches*

We compare the detection accuracy of three approaches: N-Transform, Finesse, and Odess. N-Transform and Odess generate sequence-sensitive features while Finesse generates sequence-insensitive features.



(a) Similarity distribution of the first series of datasets with MoR=0.06% and varied MoL

(b) Similarity distribution of the second series of datasets with MoL=200 and varied MoR

Fig. 9. Similarity distribution of generated pairs of similar chunks in two series of benchmark datasets generated by varying the **MoL** and **MoR**.

Three approaches estimate the similarity degree of chunks in their own way. For N-Transform and Odess, positional matching is enough, but for Finesse, a sorting is required before positional matching, as introduced in Section II-B.

Figure 10 compares three approaches on these two series of benchmark datasets. The sampling rate for Odess is configured to 1/128. All three approaches show a similar pattern of increasing error as MoR and MoL increase. Odess and N-Transform achieve very similar results in Average Estimating Error and Standard Deviation of Estimating Error, and both are lower than that for Finesse. For Finesse, when MoL increases to 800, the relationship between the size of MoL and Finesse's subchunk is subtle. Some combinations of adjusting the chunk size will align with existing sub-chunk boundaries (reduced Average Estimating Error) while other adjustments cause a 'boundary-shift' problem [28] (increased Average Estimating Error).

Therefore, the **Inference 1** in Section III-D, "compared with N-Transform, our Content-Defined Sampling approach will generate the same similarity on average", is verified in Figure 10. Meanwhile, Figure 10 also verifies that Odess has a much higher detection accuracy than Finesse.

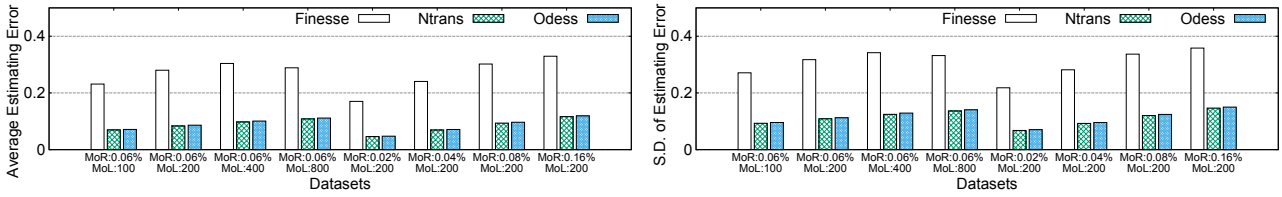
2) *Impact of Odess's Sampling Rate*

We explore the impact of the sampling rate in Odess, and the result is shown in Figure 11. In general, Odess has stable results across sampling rates, though error does increase for high sampling rates (1/512). A smaller sampling rate leads to smaller Average Estimating Error and Standard Deviation of Estimating Error, which means a better robustness; It is consistent with the Figure 6 and shows that 1/128 is a reasonable choice. Therefore, **Inference 2** in Section III-D, "the sampling rate decides the robustness of Odess", is also confirmed.

3) *Impact of the Number of Features*

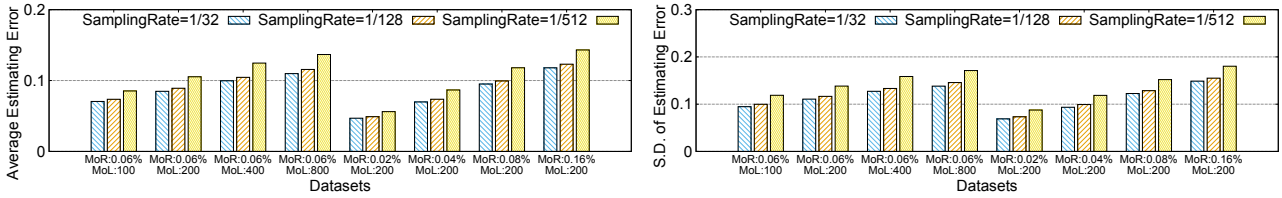
Figure 12 compares the three approaches when generating different numbers of features. We find that Odess and N-Transform perform better when number of features increases and have nearly the same Average Estimating Error and Standard Deviation of Estimating Error. This is because adding more features increases the detection accuracy of these two approaches. But Finesse does worse as the number of features increases, which is because Finesse splits chunks into a subchunk per feature and more features requires smaller sub-

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.



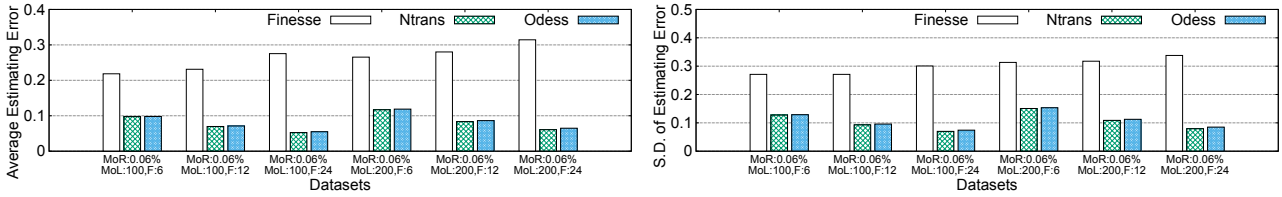
(a) Average Estimating Error (b) Standard Deviation of Estimating Error

Fig. 10. Comparison on detection accuracy of the three approaches on benchmark datasets. The sampling rate of Odess is 1/128.



(a) Average Estimating Error (b) Standard Deviation of Estimating Error

Fig. 11. Comparison on detection accuracy of Odess with different sampling rates on benchmark datasets.



(a) Average Estimating Error (b) Standard Deviation of Estimating Error

Fig. 12. Comparison on detection accuracy of the three approaches when varying features' number on benchmark datasets. Sampling rate of Odess is 1/128.

chunks. It makes the 'boundary-shift' problem more serious.

C. Experiments in Prototype Deduplication System

In this subsection, we apply our experiments on a prototype deduplication system modified to include delta compression.

The general workflow of our prototype system is shown in Figure 13. Deduplication is designed to reduce the chunk-level redundancy, and delta compression is designed to reduce redundancy between pairs of very similar but not identical chunks. Table II lists the used datasets in this subsection.

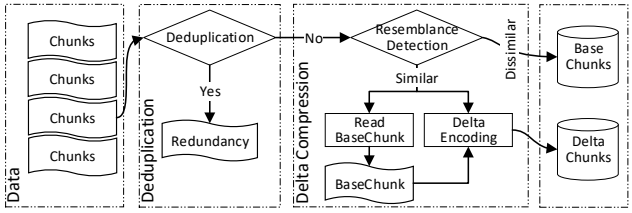


Fig. 13. General workflow of a prototype data reduction system.

We use FastCDC [38] to split data streams into chunks. In delta compression, we modified Xdelta3 [18] to only perform data reduction across similar chunks and no longer perform inner data reduction on the input chunks to better show the impact of similarity-detection and remove the impact of local compression (such as LZ [3]). Three resemblance detection methods all create 12 features and group them into 3 Super-Features, which is recommended by recent works [16], [23], [29]. A chunk may have multiple similar chunks, and we select the first matched chunk as its base chunk for delta compression, which is also known as 'FirstFit' [23]. The window sizes of rolling hash in these approaches are all configured to 32 bytes. In Finesse [14], detected chunks having considerably different sizes will be regarded as dissimilar chunks to avoid its detection weaknesses, but other works do not filter size mismatches. Thus, in this part, we avoid this limit for a high compression ratio.

1) *Data Reduction Efficiency*

First, we focus on the data reduction efficiency of the system when applying different approaches with different chunk sizes. We evaluate this experiment with three metrics, **Delta Compression Ratio (DCR)**, **Delta Compression Efficiency (DCE)** and **Similar Chunk Ratio (SCR)**. DCR reflects the total space saved by resemblance detection and then delta compression:

$$DCR = \frac{total\ size\ before\ delta\ compression}{total\ size\ after\ delta\ compression}. \quad (9)$$

DCE is used to estimate the average similarity degree between the detected similar chunks:

$$DCE = Avg.(1 - \frac{chunk\ size\ after\ delta\ compr.}{chunk\ size\ before\ delta\ compr.}). \quad (10)$$

SCR is used to study how many similar chunks will be detected by the tested approaches:

$$SCR = \frac{the\ number\ of\ detected\ similar\ chunks}{the\ number\ of\ unique\ chunks(dissimilar)}. \quad (11)$$

Table III shows the DCE, DCR, and SCR of all three approaches under different chunk sizes. N-Transform is presented as the baseline, and percentage changes are shown for

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

TABLE III
COMPARISON OF RESEMBLANCE DETECTION EFFICIENCY BETWEEN N-TRANSFORM, FINESSE AND ODESS USING DIFFERENT CHUNK SIZES

Dataset	Appr.	4 KB			8 KB			16 KB		
		DCE	DCR	SCR	DCE	DCR	SCR	DCE	DCR	SCR
TAR	Finesse	0.991 (+1.4%)	5.01 (-30.9%)	4.233 (-46.6%)	0.991 (+1.0%)	5.39 (-28.9%)	4.916 (-44.2%)	0.990 (+0.4%)	5.94 (-26.3%)	5.720 (-41.5%)
	N-Trans	0.977	7.25	7.924	0.981	7.58	8.816	0.986	8.06	9.776
	Odess	0.985 (+0.8%)	7.40 (+2.1%)	7.621 (-3.8%)	0.987 (+0.6%)	7.61 (+0.4%)	8.484 (-3.8%)	0.988 (+0.2%)	8.01 (-0.6%)	9.519 (-2.6%)
LNX	Finesse	0.998 (+0.2%)	12.5 (-37.5%)	12.26 (-48.6%)	0.999 (+0.3%)	13.8 (-37.8%)	14.23 (-50.1%)	0.999 (+0.3%)	15.5 (-38.6%)	15.94 (-51.0%)
	N-Trans	0.996	19.9	23.84	0.996	22.2	28.51	0.996	25.2	32.56
	Odess	0.995 (-0.1%)	19.2 (-3.7%)	22.00 (-7.7%)	0.996 (+0.0%)	21.6 (-2.9%)	27.09 (-5.0%)	0.996 (+0.0%)	24.8 (-1.8%)	31.91 (-2.0%)
RDB	Finesse	0.950 (+1.5%)	2.08 (-16.8%)	0.987 (-42.3%)	0.956 (+1.4%)	2.33 (-11.1%)	1.408 (-35.5%)	0.966 (+1.2%)	2.45 (-12.2%)	2.078 (-27.7%)
	N-Trans	0.936	2.50	1.712	0.943	2.62	2.182	0.955	2.79	2.874
	Odess	0.935 (-0.1%)	2.50 (+0.0%)	1.700 (-0.7%)	0.943 (+0.0%)	2.63 (+0.4%)	2.169 (-0.6%)	0.955 (+0.0%)	2.81 (+0.7%)	2.869 (-0.2%)
SYN	Finesse	0.988 (+0.9%)	1.96 (-12.1%)	0.862 (-23.6%)	0.991 (+0.7%)	2.25 (-12.5%)	1.184 (-22.0%)	0.995 (+0.5%)	2.83 (-12.4%)	1.722 (-19.9%)
	N-Trans	0.979	2.23	1.129	0.984	2.57	1.517	0.990	3.23	2.151
	Odess	0.979 (+0.0%)	2.21 (-0.9%)	1.119 (-0.9%)	0.984 (+0.0%)	2.56 (-0.4%)	1.507 (-0.7%)	0.990 (+0.0%)	3.22 (-0.3%)	2.141 (-0.5%)
VMA	Finesse	0.957 (+1.7%)	1.33 (-2.2%)	0.267 (-22.8%)	0.959 (+1.9%)	1.28 (-6.6%)	0.247 (-31.8%)	0.963 (+2.1%)	1.26 (-8.7%)	0.237 (-37.5%)
	N-Trans	0.941	1.36	0.346	0.941	1.37	0.362	0.943	1.38	0.379
	Odess	0.937 (-0.4%)	1.36 (+0.0%)	0.347 (+0.3%)	0.938 (-0.3%)	1.36 (-0.7%)	0.358 (-1.1%)	0.941 (-0.2%)	1.38 (+0.0%)	0.375 (-1.1%)
VMB	Finesse	0.952 (+1.5%)	1.20 (-0.8%)	0.155 (-20.1%)	0.955 (+1.7%)	1.19 (-2.5%)	0.149 (-25.9%)	0.960 (+1.8%)	1.19 (-4.0%)	0.146 (-30.5%)
	N-Trans	0.938	1.21	0.194	0.939	1.22	0.201	0.943	1.24	0.210
	Odess	0.935 (-0.3%)	1.22 (+0.8%)	0.195 (+0.5%)	0.937 (-0.2%)	1.22 (+0.0%)	0.199 (-1.0%)	0.941 (-0.2%)	1.23 (-0.8%)	0.206 (-1.9%)

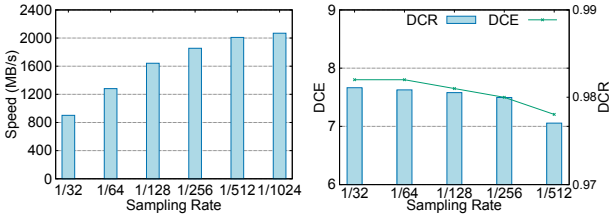
Finesse and Odess. Finesse usually reaches a higher DCE but a lower DCR and SCR, which means Finesse finds very similar chunks but misses less similar chunks. Thus the weakness of Finesse is obvious and it can not realize the potential of delta compression as much as possible.

N-Transform and Odess find more similar chunks, which can save more storage space. For these two approaches, DCR increases with the chunk size, because a bigger chunk size has more similar but not identical chunks [10].

Compared with Finesse, Odess achieves 1.230× DCR for the 4KB chunk size, 1.222× DCR for the 8KB chunk size, and 1.227× DCR for the 16KB chunk size, which is a very similar result to N-Transform.

*2) **Impact of Sampling Rate in Odess on the Speed and Detection Accuracy***

This experiment runs on the TAR dataset in Table II, and the chunk size is configured as 8KB.

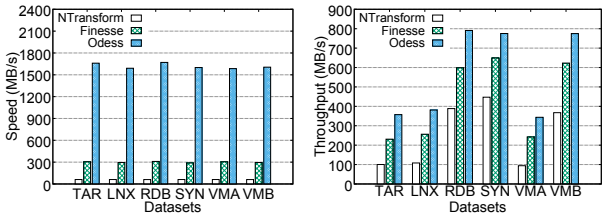


(a) Features generating speed (b) DCR & DCE

Fig. 14. Impact of sampling rate in Odess.

Figure 14(a) shows the feature generating speed of Odess using different sampling rates. The smaller the sampling rate is, the lower the computation overhead and the higher the feature generating speed will be. At the upper limit of sampling, we nearly reach the performance of Gear, which is the feature generation algorithm used before sampling and linear transforms are applied.

Figure 14(b) shows the impact of Odess's sampling rate on DCR and DCE on TAR dataset. We learn that a smaller sampling rate (i.e. 1/512) will lead to a lower DCR and DCE, which means a lower resemblance detection efficiency, and a higher sampling rate (i.e. 1/32) does the opposite. These results are also consistent with Figure 6.



(a) Features generating speed (b) System throughput

Fig. 15. Comparison between three approaches on resemblance detection and delta compression prototype systems with different datasets.

*3) **Throughput of the Whole System on Six Datasets***

We first study feature generating speed in Figure 15(a) for N-Transform, Finesse, and Odess on six datasets. Odess uses a sampling rate 1/128 based on previous experiments. Results show that N-Transform is the slowest, Finesse runs about 4.9× faster than N-Transform, and Odess runs about 5.5× faster than Finesse. Summarizing our results, Odess demonstrates a large improvement in feature generation speed.

Next, we study the influence of resemblance detection approaches on the throughput of the data reduction system in Figure 15(b). The system with Odess shows a significant advantage, achieving about 1.2× ∼ 1.54× faster throughput than the system with Finesse (average is 1.36×), and about 1.8× ∼ 3.7× faster than the system with N-Transform (average is 2.76×). Note that only non-deduplicated chunks will be processed by a resemblance detection method (as Figure 13), and from Table II we see that the deduplication ratios of RDB, SYN, and VMB are much higher than other datasets. Therefore, the throughput advantage of Odess is smaller in RDB, SYN, and VMB datasets. In Section IV-C1, we learn that Finesse identifies fewer similar chunks than N-Transform and Odess, and the system with Finesse also performs less delta encoding. But even if the system with Finesse has less computation overhead in delta encoding, the system with Odess still has a better throughput.

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.

From experiments in this subsection, we learn that compared with Finesse, a prototype data reduction system with Odess improves throughput about $1.36\times$ and increases the compression ratio about $1.22\times$; and compared with N-Transform, a prototype system with Odess improves throughput about $2.76\times$ while achieving nearly the same compression ratio.

V. Conclusion and Future Work

In this paper, we propose a fast resemblance detection approach Odess, which uses a fast rolling hash and a novel Content-Defined Sampling method. Odess is extremely fast, about $26.9\times$ faster than N-Transform at features generation for resemblance detection while maintaining an approximate detection accuracy. We applied Odess in a prototype data reduction system, and our experiments show that, compared with N-Transform, the prototype system with Odess is about $2.76\times$ faster with nearly the same compression ratio.

In the future, we will focus on improving the grouping method that is combined with our fast Content-Defined Sampling. The N-Transform approach can not specify a similarity threshold when detecting similar candidates, and chunks with lower similarity are hard to be detected.

Acknowledgments

Thanks for anonymous reviewers' insightful comments and valuable suggestions. This research was partly supported by National Key R&D Program of China under Grant no. 2018YFB1003800 and no. 2018YFB1003805; the National Natural Science Foundation of China under Grant no. 61972441, no. 61972112, and no. 61832004; the Shenzhen Science and Technology Program under Grant no. JCYJ20190806143405318, no. JCYJ20200109113427092, and no. JCYJ20170413105929681; State Key Laboratory of Computer Architecture (ICT, CAS) under Grant no. CARCHA202006. Wen Xia (xiawen@hit.edu.cn) is the corresponding author.

Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the funding agencies.

References

[1] D. Reinsel, J. Gantz, and J. Rydning, "The digitization of the world from edge to core," *IDC White Paper*, 2018.

[2] T. Asaro and H. Biggar, "Data de-duplication and disk-to-disk backup systems: Technical and business considerations," *The Enterprise Strategy Group*, pp. 2–15, 2007.

[3] J. Ziv and A. Lempel, "A universal algorithm for sequential data compression," *IEEE Trans. on information theory*, vol. 23, no. 3, pp. 337–343, 1977.

[4] ——, "Compression of individual sequences via variable-rate coding," *IEEE Trans. on Information Theory*, vol. 24, no. 5, pp. 530–536, 1978.

[5] W. Xia, H. Jiang, D. Feng, and et al., "A comprehensive study of the past, present, and future of data deduplication," *Proc. of the IEEE*, vol. 104, no. 9, pp. 1681–1710, 2016.

[6] S. Quinlan and S. Dorward, "Venti: A new approach to archival data storage," in *Proc. of FAST '02*.

[7] A. Wildani, E. L. Miller, and O. Rodeh, "Hands: A heuristically arranged non-backup in-line deduplication system," in *Proc. of IEEE ICDE'13*.

[8] L. L. You, K. T. Pollack, and D. D. E. Long, "Deep store: an archival storage system architecture," in *Proc. of IEEE ICDE'05*.

[9] F. Douglis and A. Iyengar, "Application-specific delta-encoding via resemblance detection," in *Proc. of USENIX ATC'03*.

[10] P. Shilane, G. Wallace, M. Huang, and et al., "Delta compressed and deduplicated storage using stream-informed locality," in *Proc. of HotStorage'12*.

[11] L. Xu, A. Pavlo, S. Sengupta, and et al., "Online deduplication for databases," in *Proc. of ACM SIGMOD'17*.

[12] F. Zhou, L. Zhuang, B. Y. Zhao, and et al., "Approximate object location and spam filtering on peer-to-peer systems," in *Proc. of Middleware'03*.

[13] H. Pucha, D. G. Andersen, and M. Kaminsky, "Exploiting similarity for multi-source downloads using file handprints," in *Proc. of NSDI'07*.

[14] Y. Zhang, W. Xia, D. Feng, and et al., "Finesse: Fine-grained feature locality based fast resemblance detection for post-deduplication delta compression," in *Proc. of FAST'19*.

[15] N. Jain, M. Dahlin, and R. Tewari, "Taper: Tiered approach for eliminating redundancy in replica synchronization," in *Proc. of FAST'05*.

[16] P. Shilane, M. Huang, G. Wallace, and et al., "Wan-optimized replication of backup datasets using stream-informed delta compression," *ACM Trans. on Storage*, vol. 8, no. 4, pp. 1–26, 2012.

[17] D. Gupta, S. Lee, M. Vrable, and et al., "Difference engine: Harnessing memory redundancy in virtual machines," in *Proc. of the OSDI'08*.

[18] J. MacDonald, "File system support for delta compression," Master's thesis, Department of Electrical Engineering and Computer Science, University of California at Berkeley, 2000.

[19] L. Xu, A. Pavlo, S. Sengupta, and et al., "Reducing replication bandwidth for distributed document databases," in *Proc. of SoCC'15*.

[20] L. Aronovich, R. Asher, E. Bachmat, and et al., "The design of a similarity based deduplication system," in *Proc. of SYSTOR'09*.

[21] G. Forman, K. Eshghi, and S. Chiocchetti, "Finding similar files in large document repositories," in *Proc. of SIGKDD'05*.

[22] A. Z. Broder, "On the resemblance and containment of documents," in *Proc. of Compression and Complexity of Sequences*, 1997.

[23] P. Kulkarni, F. Douglis, J. LaVoie, and et al., "Redundancy elimination within large collections of files," in *Proc. of USENIX ATC'04*.

[24] W. Xia, H. Jiang, D. Feng, and et al., "Dare: A deduplication-aware resemblance detection and elimination scheme for data reduction with low overheads," *IEEE Trans. on Computers*, vol. 65, no. 6, pp. 1692–1705, 2016.

[25] W. Xia, H. Jiang, D. Feng, and et al., "Ddelta: A deduplication-inspired fast delta compression approach," *Performance Evaluation*, vol. 79, pp. 258–272, 2014.

[26] D. A. Huffman, "A method for the construction of minimum redundancy codes," *Proc. of the IRE*, vol. 40, no. 9, pp. 1098–1101, 1952.

[27] P. Jaccard, "Etude de la distribution florale dans une portion des alpes et du jura," *Bulletin de la Societe Vaudoise des Sciences Naturelles*, vol. 37, pp. 547–579, 1901.

[28] A. Muthitacharoen, B. Chen, and D. Mazières, "A low-bandwidth network file system," *ACM SIGOPS Operating Systems Review*, vol. 35, pp. 174–187, 2001.

[29] X. Lin, G. Lu, F. Douglis, and et al., "Migratory compression: Coarse-grained data reordering to improve compressibility," in *Proc. of FAST'14*.

[30] A. Z. Broder, "Identifying and filtering near-duplicate documents," *Lecture Notes in Computer Science*, vol. 1848, pp. 1–10, 1998.

[31] M. O. Rabin, "Fingerprinting by random polynomials," *Technical report*, 1981.

[32] M. Fu, D. Feng, Y. Hua, and et al., "Design tradeoffs for data deduplication performance in backup workloads," in *Proc. of FAST'15*.

[33] M. Lillibridge, K. Eshghi, D. Bhagwat, and et al., "Sparse indexing: Large scale, inline deduplication using sampling and locality," in *Proc. of the FAST'09*.

[34] D. Meister, J. Kaiser, and A. Brinkmann, "Block locality caching for data deduplication," in *Proc. of SYSTOR'13*.

[35] G. Wallace, F. Douglis, H. Qian, and et al., "Characteristics of backup workloads in production systems," in *Proc. of FAST'12*.

[36] W. Xia, H. Jiang, D. Feng, and et al., "Silo: A similarity-locality based near-exact deduplication scheme with low ram overhead and high throughput," in *Proc. of USENIX ATC'11*.

[37] B. Zhu, K. Li, and H. Patterson, "Avoiding the disk bottleneck in the data domain deduplication file system," in *Proc. of FAST'08*.

[38] W. Xia, Y. Zhou, H. Jiang, and et al., "Fastcdc: A fast and efficient content-defined chunking approach for data deduplication," in *Proc. of USENIX ATC'16*.

[39] F. Ni and S. Jiang, "Rapidcdc: Leveraging duplicate locality to accelerate chunking in cdc-based deduplication systems," in *Proc. of the ACM SoCC'19*.

Authorized licensed use limited to: University of Canberra. Downloaded on November 14,2022 at 15:37:18 UTC from IEEE Xplore. Restrictions apply.